

Hướng Dẫn Đây Đủ Spec-Driven Development (SDD)

Mục Lục

- 1. Tổng Quan
 - 2. Khái Niệm Cốt Lõi
 - 3. Quy Trình SDD
 - 4. Ví Dụ Thực Tế
 - 5. Làm Việc Với Team
 - 6. Quản Lý Thay Đổi
 - 7. Best Practices
 - 8. Tools và Resources
-

1. Tổng Quan

1.1 Spec-Driven Development là gì?

Spec-Driven Development (SDD) là phương pháp phát triển phần mềm trong đó:

- **Specifications** (bản đặc tả) là nguồn sự thật trung tâm
- **Code phục vụ specifications**, không phải ngược lại
- **AI/Coding agents** có thể generate implementation từ specifications

1.2 Tại sao cần SDD?

Vấn đề với phương pháp truyền thống:

- Khoảng cách giữa specification và implementation
- Code trở thành nguồn sự thật, specification bị bỏ quên
- Khó maintain và scale khi requirements thay đổi
- Technical debt tích lũy theo thời gian

Lợi ích của SDD:

-  **Intent-driven:** Focus vào "WHAT" và "WHY", không phải "HOW"
-  **Flexibility:** Dễ dàng pivot và thay đổi
-  **AI-friendly:** Coding agents hiểu và generate code chính xác
-  **Traceability:** Mọi code đều trace back về requirements
-  **Reduced technical debt:** Regenerate code từ specs

1.3 Khi nào nên dùng SDD?

Đặc biệt hiệu quả cho:

1. **Greenfield projects** (zero-to-one)
2. **Legacy modernization**
3. **Rapid prototyping**
4. **Projects với requirements thay đổi thường xuyên**

2. Khái Niệm Cốt Lõi

2.1 Specification

Specification mô tả "**CÁI GÌ**" (what) và "**TẠI SAO**" (why) của phần mềm.

Specification bao gồm:

markdown

- Business requirements
- User stories và use cases
- Business rules và constraints
- Expected behaviors
- Success metrics
- Non-functional requirements

Specification KHÔNG bao gồm:

- markdown
- Implementation details
 - Technology choices
 - Code structure
 - Database schemas
 - API designs

2.2 Implementation Plan

Plan mô tả "NHƯ THẾ NÀO" (how) để xây dựng phần mềm.

Plan bao gồm:

- markdown
- Technology stack
 - System architecture
 - Database design
 - API structure
 - Security approach
 - Performance strategy
 - Development phases

2.3 Task Breakdown

Tasks là các **action items cụ thể** để thực hiện plan.

Mỗi task cần có:

markdown

- Priority level
- Time estimate
- Dependencies
- Acceptance criteria
- Assigned team/person

3. Quy Trình SDD

3.1 Overview

Business Need → Specification → Implementation Plan → Task Breakdown
→ Code Generation/Implementation → Testing & Validation → Deployment

Change Request → Back to Specification (iterate)

3.2 Chi Tiết Từng Bước

Bước 1: Tạo Specification

markdown

[Product Name] Specification

Vision

[Mô tả tầm nhìn sản phẩm]

User Personas

- [Persona 1]: [Description]
- [Persona 2]: [Description]

Core Features

Feature 1

- User story
- Acceptance criteria
- Business rules

Non-functional Requirements

- Performance
- Security
- Scalability

Bước 2: Tạo Implementation Plan

markdown

```
# Implementation Plan

## Technical Architecture
### Technology Stack
- Frontend: [Technologies]
- Backend: [Technologies]
- Database: [Technologies]

### System Design
[Architecture diagram hoặc description]

### Database Schema
[Table designs]

### API Design
[Endpoint structure]

## Development Phases
### Phase 1: Foundation
### Phase 2: Core Features
### Phase 3: Polish
```

Bước 3: Tạo Task Breakdown

markdown

```
# Task Breakdown

## Sprint 1: [Sprint Name]

### Task 1.1: [Task Name]
**Priority**: High/Medium/Low
**Estimate**: X hours
**Dependencies**: None/[Task IDs]
**Acceptance Criteria**:
- [ ] Criteria 1
- [ ] Criteria 2
```

Bước 4: Implementation

Sử dụng AI coding agents hoặc development team để implement từng task.

4. Ví Dụ Thực Tế

4.1 Recipe Sharing Platform - Complete Flow

SPECIFICATION

markdown

Recipe Sharing Platform Specification

Vision

Create a community-driven platform where home cooks can share, discover, and organize recipes with a focus on simplicity and visual appeal.

Core User Needs

- Share personal recipes with beautiful photos
- Discover recipes based on ingredients they have
- Save and organize favorite recipes
- Scale recipe portions automatically
- Share shopping lists with family members

Key Features

Recipe Creation & Management

Users can create recipes with:

- Recipe title and description
- Ingredients list with quantities and units
- Step-by-step instructions with optional photos
- Prep time and cooking time
- Difficulty level (Easy/Medium/Hard)
- Category tags (breakfast, vegan, gluten-free, etc.)
- Number of servings

Recipe Discovery

Users can find recipes through:

- Search by recipe name or ingredients
- Filter by dietary restrictions
- Filter by cooking time

- Browse by category
- "What can I make?" feature
- Trending recipes based on community activity

Social Features

- Follow other cooks
- Like and comment on recipes
- Share recipes to social media
- Rate recipes (5-star system)
- Save recipes to personal collections

Smart Features

- Automatic portion scaling
- Generate shopping list from selected recipes
- Share shopping lists via link or email
- Nutrition information auto-calculation
- Recipe print view optimized for kitchen use

Business Rules

- Recipes must have at least title, one ingredient, and one step
- Users must be registered to create recipes
- Images limited to 5MB per upload
- Maximum 20 images per recipe

Success Metrics

- Users can create a recipe in under 5 minutes
- Search results return in under 1 second
- Mobile-responsive design works on all devices

IMPLEMENTATION PLAN

markdown

Recipe Platform - Implementation Plan

Technical Architecture

Technology Stack

Frontend:

- React 18 with TypeScript
- Tailwind CSS for styling
- React Query for data fetching
- React Hook Form for forms

Backend:

- Node.js with Express
- PostgreSQL database
- Redis for caching
- AWS S3 for image storage

Database Schema

-- Core Tables

users (id, email, username, password_hash, created_at)

recipes (id, user_id, title, description, prep_time, cook_time, servings, diffi

ingredients (id, recipe_id, name, quantity, unit, order_index)

instructions (id, recipe_id, step_number, description, image_url)

categories (id, name, slug)

recipe_categories (recipe_id, category_id)

favorites (user_id, recipe_id, created_at)

ratings (user_id, recipe_id, rating, created_at)

API Design

```
# Authentication
POST /api/auth/register
POST /api/auth/login
GET  /api/auth/me

# Recipes
GET  /api/recipes      # List with filters
GET  /api/recipes/:id  # Single recipe
POST /api/recipes      # Create recipe
PUT  /api/recipes/:id  # Update recipe
DELETE /api/recipes/:id # Delete recipe

# Social
POST /api/recipes/:id/favorite
POST /api/recipes/:id/rate
POST /api/recipes/:id/comment

### Development Phases

**Phase 1: Foundation (Week 1-2)**
- Setup development environment
- Database design and setup
- Authentication system
- Basic API structure

**Phase 2: Core Features (Week 3-4)**
- Recipe CRUD operations
- Image upload system
- Search functionality
```

- **Phase 3: Social Features (Week 5)****

 - Favorites, ratings, comments
 - User profiles

****Phase 4: Smart Features (Week 6)****

 - Portion scaling
 - Shopping list generation

TASK BREAKDOWN

markdown

```
# Recipe Platform - Task List

## Sprint 1: Foundation Setup

### Task 1.1: Development Environment
**Priority**: High
**Estimate**: 2 hours
**Dependencies**: None
**Description**:
- Initialize Git repository
- Setup Node.js project with TypeScript
- Configure ESLint and Prettier
**Acceptance Criteria**:
- [ ] npm run dev starts server
- [ ] Linting rules enforced

### Task 1.2: Database Setup
**Priority**: High
**Estimate**: 3 hours
**Dependencies**: Task 1.1
**Description**:
- Create PostgreSQL database schema
- Write migration files
- Setup Prisma ORM
**Acceptance Criteria**:
- [ ] All tables created
- [ ] Migrations run successfully

### Task 1.3: Authentication Backend
**Priority**: High
```

```
**Estimate**: 4 hours
**Dependencies**: Task 1.2
**Description**:
- Implement JWT authentication
- Create register/login endpoints
**Acceptance Criteria**:
- [ ] Users can register
- [ ] Users can login
- [ ] Protected routes work

## Sprint 2: Core Recipe Features

### Task 2.1: Recipe API Endpoints
**Priority**: High
**Estimate**: 6 hours
**Dependencies**: Task 1.3
**Description**:
- Create CRUD endpoints for recipes
- Add authorization checks
**Acceptance Criteria**:
- [ ] Can create/read/update/delete recipes
- [ ] Only owners can edit their recipes

### Task 2.2: Image Upload System
**Priority**: High
**Estimate**: 5 hours
**Dependencies**: Task 2.1
**Description**:
- Setup AWS S3 bucket
- Create image upload endpoint
- Implement image resizing
```

****Acceptance Criteria**:**

- [] Images upload to S3
- [] Images are resized
- [] Multiple images per recipe work

4.2 Các Template Specification Khác

Todo App (Simple)

markdown

Todo List Application Specification

Purpose

Create a simple task management application for daily activities.

Core Requirements

User Stories

- As a user, I want to add new tasks
- As a user, I want to mark tasks as complete
- As a user, I want to edit tasks
- As a user, I want to delete tasks
- As a user, I want to filter tasks by status

Business Rules

- Tasks must have a title (max 200 chars)
- Tasks can have optional description (max 1000 chars)
- Tasks can have optional due date
- Completed tasks show strikethrough
- Tasks persist between sessions

E-commerce Product Catalog

markdown

Product Catalog System Specification

Overview

Product browsing and search system for customers.

Core Functionality

Product Discovery

- Category browsing with hierarchy
- Text search with auto-suggestions
- Filter by price, brand, ratings
- Sort by relevance, price, ratings

Product Display

- Multiple images with zoom
- Detailed specifications
- Price and discounts
- Ratings and reviews
- Stock availability
- Estimated delivery

Business Rules

- Out-of-stock items remain visible
- Prices update real-time
- Search results < 2 seconds
- Reviews require purchase verification

5. Làm Việc Với Team

5.1 Team Structure

markdown

Team Organization for SDD

Roles & Responsibilities

Product Owner

- Owns business requirements
- Approves specifications
- Prioritizes features

Tech Lead

- Reviews technical plans
- Approves architecture decisions
- Assigns tasks

Backend Team

- Owns API design
- Implements business logic
- Manages database

Frontend Team

- Owns UI/UX implementation
- Implements client logic
- Manages user interactions

QA Team

- Creates test scenarios from specs
- Validates acceptance criteria
- Reports bugs against specs

5.2 Backend + Frontend Collaboration

Shared Specification Structure

```
markdown

# [Project] Specification

## 🎯 Business Requirements
[Both teams read this]

## 📱 User Interface Requirements
[Frontend team owns this]

## 🔧 API Requirements
[Backend team owns this]

## 🤝 Integration Points
[Both teams review this]
```

API Contract First

```
yaml
```

```
# API Contract Definition
POST /api/auth/login
Request:
  email: string
  password: string
Response 200:
  token: string
  user: object
Response 401:
  error: string
```

Parallel Development Pattern

markdown

Week 1 Sprint Plan

Backend Tasks

- B1.1: Setup project [2h]
- B1.2: Create auth API [4h]
- B1.3: Create user CRUD [3h]

Frontend Tasks

- F1.1: Setup project [2h]
- F1.2: Create auth forms [4h]
⚠ Blocked by B1.2
- F1.3: Create user pages [3h]
⚠ Blocked by B1.3

Integration Points

- Day 3: Auth API ready → Frontend integration
- Day 4: User API ready → Frontend integration

5.3 Communication Patterns

markdown

Daily Sync Template

Backend Status

- ✅ Completed: Auth APIs
- 🔄 In Progress: User APIs (50%)
- 🚫 Blocked: Need AWS credentials

Frontend Status

- ✅ Completed: Login form
- 🔄 In Progress: User profile (mock data)
- ⌚ Waiting: User APIs for integration

Integration Issues

- 🐛 CORS error on /api/login
- 📝 Need pagination in /api/users

6. Quản Lý Thay Đổi

6.1 Change Classification

markdown

Change Classification Matrix

🟢 Minor Changes

Examples: UI text, colors, validation rules

→ Action: Update task directly

🟡 Moderate Changes

Examples: New fields, new endpoints, new features

→ Action: Append to spec + Add new tasks

🔴 Major Changes

Examples: Architecture change, data model change

→ Action: New version of spec + Re-plan

6.2 Change Request Process

markdown


```
# Change Request Template

## Request Information
**ID**: CR-2024-001
**Date**: 2024-01-15
**Requested By**: [Name]
**Type**: [ ] Minor [X] Moderate [ ] Major

## Description
[What needs to change and why]

## Impact Analysis
### Affected Components
- Backend: [List impacts]
- Frontend: [List impacts]
- Database: [List impacts]

### Effort Estimate
- Backend: X hours
- Frontend: Y hours
- Testing: Z hours

## Decision
**Status**: [ ] Pending [X] Approved [ ] Rejected
**Approved By**: [Name]
**Target Sprint**: Sprint X
```

6.3 Version Control Strategy

```
specifications/  
├─ v1.0.0/  
│   ├── spec.md  
│   ├── plan.md  
│   └─ tasks.md  
├─ v1.1.0/  
│   ├── spec.md  
│   ├── plan.md  
│   ├── tasks.md  
│   └─ changelog.md  
└─ v2.0.0/  
    ├── spec.md  
    ├── plan.md  
    ├── tasks.md  
    └─ migration-guide.md
```

6.4 Handling Changes During Development

Scenario: Adding PayPal to existing Stripe payment

markdown

```
## Original State (Week 3)
Task 3.1: Stripe integration [✅ Done]
Task 3.2: Checkout with credit card [🔄 50% done]

## Change Request
"Add PayPal payment option"

## Decision
- Keep existing work? → Yes
- Additional or replacement? → Additional
- Impact? → +3 days

## Action
### Keep Original Tasks
Task 3.1: Stripe integration [✅ Keep as-is]
Task 3.2: Checkout with credit card [Continue to 100%]

### Add New Tasks
Task 3.5: 🆕 PayPal SDK integration [4h]
Task 3.6: 🆕 Payment method selector [2h]
Task 3.7: 🆕 PayPal webhook handling [3h]

### Update Specification
Version: 1.0.0 → 1.1.0
Added: Section "PayPal Payment Integration"
```

7. Best Practices

7.1 Writing Good Specifications

markdown

DO

- Focus on user needs and business value
- Use clear, unambiguous language
- Include acceptance criteria
- Define success metrics
- Consider edge cases

DON'T

- Include technical implementation details
- Specify technology choices
- Design database schemas
- Define code structure
- Make assumptions about HOW

7.2 Creating Effective Plans

markdown

Plan Checklist

- [] Technology stack justified
- [] Architecture scalable
- [] Security considered
- [] Performance targets defined
- [] Database normalized
- [] API RESTful/GraphQL consistent
- [] Error handling defined
- [] Monitoring strategy included

7.3 Task Management

markdown

Good Task Definition

- ✓ Clear title and description
- ✓ Realistic time estimate
- ✓ Dependencies identified
- ✓ Acceptance criteria measurable
- ✓ Priority assigned
- ✓ Can be completed in 1-2 days

Task Sizing Guide

- Small: 1-4 hours
- Medium: 4-8 hours (1 day)
- Large: 8-16 hours (2 days)
- Too Large: Break down further

7.4 Documentation Standards

markdown

```
## Every Document Should Have
- Version number
- Date created/modified
- Author/Owner
- Status (Draft/Review/Approved)
- Change log (for updates)
```

```
## File Naming Convention
[project]-[type]-[version].md
Examples:
- ecommerce-spec-v1.0.0.md
- ecommerce-plan-v1.0.0.md
- ecommerce-tasks-sprint1.md
```

8. Tools và Resources

8.1 Spec Kit (GitHub)

```
bash
```

```
# Install Spec Kit
npm install -g @github/spec-kit

# Initialize project
uvx --from git+https://github.com/github/spec-kit.git specify init PROJECT_NAME

# Create specification
/specify "Build an app that..."

# Generate plan
/plan

# Create tasks
/tasks

# Execute with AI agent
/execute task-1.1
```

8.2 Project Structure

```
project/
├── .github/
│   ├── prompts/
│   │   ├── specify.prompt.md
│   │   ├── plan.prompt.md
│   │   └── tasks.prompt.md
├── specifications/
│   ├── current/
│   │   ├── spec.md
│   │   ├── plan.md
│   │   └── tasks.md
│   └── archive/
│       └── v1.0.0/
├── docs/
│   ├── api-contracts/
│   ├── change-requests/
│   └── decisions/
├── src/
│   ├── backend/
│   └── frontend/
└── tests/
```

8.3 AI Coding Agents

markdown

Compatible Tools

- GitHub Copilot
- Claude Code
- Cursor
- Gemini CLI
- Continue.dev

Usage Pattern

1. Feed specification to AI
2. AI generates implementation plan
3. Review and approve plan
4. AI creates task breakdown
5. AI implements each task
6. Human reviews and tests

8.4 Templates Repository

markdown

Starter Templates

- SaaS Application
- E-commerce Platform
- Mobile App Backend
- Real-time Chat System
- Analytics Dashboard
- CMS Platform

Each Template Includes

- Sample specification
- Implementation plan
- Task breakdown
- API contracts
- Database schemas
- Test scenarios

Tài Liệu Tham Khảo

- [GitHub Spec Kit Repository](#)
- [Spec-Driven Development Blog Post](#)
- [Anthropic Claude Documentation](#)
- [API Design Best Practices](#)

Kết Luận

Spec-Driven Development không chỉ là một methodology mới, mà là một cách tư duy mới về phát triển phần mềm trong thời đại AI:

- **Specifications trở thành executable** thay vì chỉ là documentation
- **Intent là nguồn sự thật**, không phải code

- **AI augments human creativity**, không thay thế
- **Flexibility và maintainability** được tối ưu hóa

Bắt đầu với projects nhỏ, học từ kinh nghiệm, và scale up dần dần. SDD sẽ giúp team của bạn deliver nhanh hơn, linh hoạt hơn, và với ít technical debt hơn.

Document Version: 1.0.0

Last Updated: 2024

Status: Complete

Author: SDD Guidelines Team

Content is user-generated and unverified.